1-amaliy mashg’ulot.

Mavzu: NoCoding texnologiyasida tanlangan mavzu asosida loyiha yaratish. Git bilan ishlash.

Ishdan Maqsad. Talabalarga Nocoding texnologiyasidan foydalanish haqida ma’lumot bering. Ularga Git bilan ishlashni tushuntirish.

Kerakli jihozlar. 1. NoCoding platformalari (tanlov asosida): Wix, Webflow, Bubble, Tilda, Carrd; 2. Git dasturi**:** Git bash yoki GitHub Desktop (GitHub yoki GitLab hisob qaydnomasi zarur). 3. Veb-brauzer**:** Google Chrome yoki Firefox.

Nazariy qism.

So‘nggi yillarda dasturiy ta’minot yaratish sohasida katta inqiloblar sodir bo‘lmoqda. Ulardan biri — No Code texnologiyalari. Ushbu yondashuv dasturlashni bilmasdan ham ilovalar, veb-saytlar va boshqa dasturlar yaratish imkonini beradi. No Code — texnologiyalar va oddiy foydalanuvchilar o‘rtasidagi farqni bartaraf etuvchi innovatsion yondashuv.

No Code nima?

No Code — bu dasturlar yaratish uchun kod yozishni talab qilmaydigan platformalardir. Ular grafik interfeyslar, shablonlar va «drag-and-drop» vositalar yordamida ishlaydi. No Code platformalari har xil funksiyalarni birlashtirish, ma’lumotlarni boshqarish va loyihalarni tezda ishlab chiqish imkonini beradi.

No Code platformalarining misollari:

Webflow — murakkab veb-saytlarni oson yaratish.

Bubble — dasturlar yaratish uchun universal vosita.

Airtable — ma’lumotlar bazasi va boshqaruv vositasi.

Zapier — turli xizmatlarni avtomatlashtiruvchi vosita.

No Code platformalarining afzalliklari

1. Tezkorlik. No Code yordamida dasturlarni tez ishlab chiqish mumkin. Startaplar va kichik bizneslar o‘z loyihalarini bozorda tezroq ishga tushirish imkoniyatiga ega bo‘ladi.

2. Texnik bilim talab etilmaydi. No Code platformalari dasturlashni bilmagan foydalanuvchilar uchun ham oson. Ular grafik interfeys orqali barcha jarayonlarni boshqarishi mumkin.

3. Xarajatlarni kamaytiradi. Texnik jamoani yollash yoki murakkab infratuzilma qurish uchun katta mablag‘ talab qilinmaydi. No Code kichik byudjet bilan ham muvaffaqiyatli ishlashga yordam beradi.

4. Tajriba uchun qulay. No Code dasturlarini foydalanib, g‘oyalarni sinovdan o‘tkazish juda oson. Minimal xarajat bilan prototip yaratish va uni test qilish mumkin.

5. Moslashuvchanlik. Ko‘plab platformalar turli xizmatlar va vositalar bilan integratsiyaga ega, masalan, API orqali boshqa tizimlar bilan ishlash.

No Code platformalarining kamchiliklari

1. Cheklangan funksionallik. Maxsus talablar yoki murakkab funktsiyalarni amalga oshirishda No Code platformalari yetarli emas bo‘lishi mumkin.

2. Moslashuvchanlikdagi muammolar. Platformaning imkoniyatlari va cheklovlariga bog‘liq bo‘lganlik tufayli, foydalanuvchi barcha jarayonlarni to‘liq nazorat qila olmaydi.

3. Ishlash unumdorligi. Ba’zi hollarda, katta hajmdagi ma’lumotlar yoki murakkab algoritmlar bilan ishlash samaradorligi past bo‘lishi mumkin.

4. Platformaga bog‘liqlik. No Code texnologiyasida yaratilgan ilovalar ko‘pincha ma’lum bir platformaga bog‘liq bo‘lib qoladi. Bu uzoq muddatda muammolar keltirib chiqarishi mumkin.

No Code qachon foydali?

No Code platformalari quyidagi holatlarda eng foydali:

Kichik biznes va startaplar;

G‘oyani tezda sinovdan o‘tkazish zarurati;

Vaqt va byudjet cheklovlari;

Texnik ko‘nikmasiz foydalanuvchilar uchun.

No Code platformalarining kelajagi

No Code texnologiyasi hozirda katta rivojlanish bosqichida. Bu platformalar:

Katta hajmdagi loyihalar uchun moslashuvchanroq bo‘lib boradi;

Sun’iy intellekt va avtomatlashtirish texnologiyalari bilan integratsiyalashadi;

Dasturlash bo‘yicha bilim talabini yanada kamaytiradi.

Kelajakda No Code texnologiyalari dasturlashning an’anaviy usullarini qisman almashtirishi va har bir insonni o‘zining ijodkor dasturchisiga aylantirishi mumkin.

GitHub nima va u nimasi bilan Git’dan farq qiladi?

Git Hub bu – AyTi loyihalarni hosting qilish va birgalikda dasturlash uchun yaratilgan bulut platforma bo‘lib, uning negizida versiyalarni nazorat qilishning mashhur tizimi — Git, shuningdek, dasturchilar uchun to‘laqonli ijtimoiy tarmoq yotadi.

Bu yerdan turli dasturlash tillaridagi ko‘plab open-source loyihalarni topish, ularda ishtirok etish, rezyumega havola qo‘yish uchun kod namunalari joylangan portfoliongizni yuklash, ochiq loyihalardagi qiziq arxitekturaviy yechimlarni ko‘rish, tajribali dasturchilar qanday kod yozishini kuzatish, shuningdek, dasturlash uchun kerak bo‘ladigan ko‘plab foydali va bepul uskunalarni yuklab olish mumkin. Aytgancha, ba’zi uddaburonlar GitHub’da dasturlardan tashqari, haqiqiy kitob va maqolalardan iborat butun boshli kutubxonalarni tashkil qilishni ham eplaydi.

Agar siz istalgan ochiq dasturdagi qandaydir xususiyatdan norozi bo‘lsangiz va bu dastur GitHub’da joylangan bo‘lsa, loyiha izohlarida fikringizni bildirishingiz mumkin. Eng yaxshisi issue (buning nimaligini keyinroq aytamiz) rasmiylashtirib, muammoni mustaqil tarzda to‘g‘rilab qo‘yishingiz mumkin. Yaxshi ishlangan ochiq loyihalar mualliflariga donat yoki oddiy iliq so‘zlar bilan ham minnatdorlik bildirishni unutmang. Ular bundan xursand bo‘ladi.

Deyarli istalgan AyTi kompaniyada kod qayerdadir saqlanadi, ko‘p hollarda esa ana shu “qayerdir” aynan GitHub bo‘ladi. GitHub’ning anchagina mashhur raqibi

ham bor, bu ham Git asosidagi GitLab bo‘lib, ular turli kompaniyalarning platformasi bo‘lsa-da, faoliyati juda o‘xshash.

GitHub va Git’ni adashtirmaslik kerak. GitHub Git versiyalar nazorati tizimining realizatsiyalaridan bittasi (grafik interfeysli Git-mijozlarning qarasangiz buni his qilasiz), unga ko‘plab qulay uskuna va imkoniyatlar (izohlar, issues, giperhavolalar, matnni formatlash va boshqalar) qo‘shilgan. GitHub’ni Git haqida bilimga ega bo‘lmasdan ishlatish mumkinligini ham yodda saqlang (aksinchasi ham mumkin).

Ajoyib yangrayapti, to‘g‘rimi? Endi hamma narsani hal qilish va sizga u kerakligini aniqlash uchun GitHub’dan foydalanish haqidagi qo‘llanmamizni o‘qishga kirishing.

Sizga ham GitHub kerakligini qanday bilsa bo‘ladi?

Albatta, GitHub hammaga ham kerak emas. Agar siz dasturlashni endigina boshlagan bo‘lsangiz va shaxsiy foydalanish maqsadida kichik loyiha bajarayotgan bo‘lsangiz, loyihani lokal mashinada saqlash ham sizga to‘g‘ri keladi. Ehtimol, endigina o‘zingizga yoqqan dasturlash tilini o‘rganayotgandirsiz va hamma narsani birdan sinab ko‘rishni istamassiz.

Birinchi navbatda GitHub tez-tez yangilik kiritiladigan, versiyalari ko‘p, fayllar soni katta, dasturlashni sinxronlashtirish va dasturni qulay yangilash talab qilinadigan loyihalar uchun kerak bo‘ladi.

Ammo bunda ham istisno holatlar bor – ba’zi Linux yadrosining eng muhim dasturchilari hamon kod yangilanishlarini elektron pochta orqali fayl arxivlari shaklida almashadi.

Manba kodlarini saqlashning chindan ham ko‘plab boshqa usullari bor: ular uchun “Mening kompyuterim” bo‘limida papka yaratish, ularni bulutga yuklab, versiyalarni yozib qo‘yish, hatto Telegram yoki VKontakte’dagi “Saqlangan xabarlar”da ham saqlash mumkin (biroz qiyin tuyulishi mumkin, ammo amalga oshirsa bo‘ladi).

Shuningdek, o‘zgarishlar ro‘yxatini ~~telefon yoki muzlatkichda~~ qayd holida yoki yopiq Telegram kanalda matn holida saqlash ham mumkin. Loyihani dasturingiz fayllaridan iborat ZIP–arxivni yuklash va ochish orqali ham deploy’ni amalga oshirsa

bo‘ladi (ayniqsa, maqsadingiz “noutbuk bilan yordamga kelgan” do‘stingiz yoki dugonangizga shunchaki dasturni ko‘rsatishdan iborat bo‘lsa). Yuqoridagi usullarning har biri o‘zicha yaxshi, ammo AyTida ishlash uchun GitHub’ga ko‘nikish kerak: bu shu sohaning standarti.

Qiziqarli fakt: yaqinda GitHub’ning ruscha muqobili GitFlic ham paydo bo‘ldi. Servis jamoasining xabar berishicha, ularning maqsadi “mahalliy operatsion tizim, dastur, ilova va server yechimlarini ishlab chiqish uchun yangi to‘lqin hosil qilish” ekan. Uning o‘ziga xosliklaridan biri bu — Telegram bilan integratsiya qilingani.

GitHub imkoniyatlarining katta qismi bepul, ammo platformada pulli tariflar ham bor. Ularga uzoq vaqt duch kelmasligingiz ham mumkin, chunki oddiy dasturchi uchun bepul imkoniyatlar ham yetib ortadi.

Bu Fray. U GitHub’dan foydalana olmadi va kelajakka uchib ketdi. Fray kabi bo‘lmang

GitHub’ning asosiy konsepsiyalari sodda tilda

Yangi foydalanuvchilar GitHub interfeysini tushunmasligi mumkin. Platforma paydo bo‘lganidan beri ko‘p yangi uskunalarga ega bo‘ldi, ammo eng asosiysi o‘zgarmasdan qolgan.

Deyarli barcha terminologiya Git’dan olingan. Repozitoriy, tarmoq, kommit, fork kabilar asosiy atamalardan. Bu nomlardan ba’zilarini tanlash (ingliz tilini bilgan taqdiringizda ham) unchalik mos emasdek ko‘rinishi mumkin, ammo shunday bo‘lib qolgan.

Fayllarni qanday yuklash, repozitoriylarni yaratish va boshqa amallarni keyingi bo‘limda ko‘rib chiqamiz, shuning uchun ba’zi atamalardagi turli harakatlarni ko‘rganda qo‘rqib o‘tirmang. Hammasini rasmlar bilan ko‘rsatamiz.

Repozitoriy

Bu dasturingiz ichki direktoriya va fayllaridan iborat asosiy papkasi bo‘lib, ayni damda o‘sha papkaning GitHub’dagi sahifasi hamdir. Repozitoriyga hamma narsani yuklash mumkin, ammo unda manba kodi mavjud fayllar va qandaydir qo‘shimcha materiallar, masalan, GUI yoki sahifalash uchun kerak bo‘ladigan grafika (rasmlar, ikonkalar va hokazo) saqlanishiga mo‘ljallangan.

Repozitoriylar omma uchun ochiq yoki yopiq bo‘lishi, ularning ichida boshqa papkalar yaratib, versiyalardagi o‘zgarishlarni kuzatib borish mumkin. Repozitoriylarni to‘g‘ridan to‘g‘ri sayt interfeysida turib, buyruq satrida, GitHub kompyuter dasturida yoki servis bilan integratsiyani quvvatlaydigan turli dasturlash vositalari (IDE) kabilar orqali boshqarish mumkin.

Tarmoq (branch)

Tarmoqlarda o‘zgarish va yangilanishlar guruhlanadi. Masalan, bitta asosiy branch (standart holatda main branch) va bitta beta branch yaratiladi. Tarmoqlar bir-biridan mustaqil bo‘ladi, ammo xohish tug‘ilsa, kodda farq bo‘lsa ham ularni birlashtirish (merge – birlashish) mumkin.

Repozitoriyni o‘zgartirish usullari: kommit, push, klon, fork

Repozitoriyga o‘zgartirishlarni to‘g‘ridan to‘g‘ri yoki uning nusxasini yaratib kiritish mumkin. O‘zgartirish kiritishning o‘zi “kommit” (inglizcha commit – amalga oshirmoq) deyiladi, uning vaqtinchalik belgisi va xesh-summasi bo‘ladi.

Kommit o‘zgarishlarni lokal repozitoriy (kompyuteringizdagi) dan masofaviy (remote repository, ya’ni GitHub’dagi) repozitoriyga o‘tkazish push (inglizcha push – itarish), so‘zma so‘z tarjimada esa o‘zgartirishlarni itarib kiritish deyiladi.

O‘zgartirish kiritish uchun repozitoriyni nusxalash ikki usulda amalga oshiriladi:

klonlash (clone) – shunchaki lokal kompyuter yoki serverga nusxalash;

forklash (inglizcha fork – tarmoqlash) – ishni “boshqa branch yo‘lida davom etish” uchun repozitoriyning alohida nusxasini yaratish.

Agar siz begona loyihaning muallifiga aniq yaxshilanishlarni taklif qilish maqsadida forklagan bo‘lsangiz, ishingiz tayyor bo‘lganda uni manba repozitoriyga “pull qilish”, ya’ni pull request (o‘zgartirish kiritishga ruxsat so‘rash) kerak bo‘ladi.

Yuqoridagilar kerakli faktlarning 90 foizini tashkil qiladi. Yanada zerikarliroq batafsil ma’lumotlar hujjatlar va GitHub’ga o‘rgatish bo‘limida, shuningdek, Git bo‘yicha qo‘llanmaning o‘zida ham keltirilgan. Biz esa ularni amalda qo‘llashga urinib ko‘ramiz.

Repozitoriylar yaratish va fayllar yuklash

GitHub’dan foydalanishning eng oddiy usuli sayt orqali, shuning uchun biz ham saytdan boshlaymiz.

Repozitoriylar bilan ishlashdagi eng oddiy harakat bu – fayl yaratish va uni yuklash bo‘lib, uni avvalroq ko‘rib chiqdik. Har ikki ish 30 soniyadan ortiq vaqtingizni olmaydi.

O‘tgan safargi material xulosalarini qisqacha takrorlab o‘tamiz:

Repozitoriy yaratish uchun saytning yuqori o‘ng burchagidagi + belgisiga bosib, **New repository** punktini tanlash, nom va tavsifni to‘ldirib, kerakli joylarni belgilab, **Create repository** tugmasini bosish kerak;

Fayllarni yuklash uchun kerakli repozitoriyga kirib, **Add file** tugmasiga bosib, **Upload files** tugmasini bosish kerak.

1-qadam. Veb-versiyada GitHub-repozitoriy yaratish

2-qadam. GitHub-repozitoriy yaratish sahifasi

Lekin GitHub bilan ishlashni o‘rganish uchun dasturlash davomida kerak bo‘ladigan boshqa harakatlar: klonlash/fork, branchlarni birlashtirish, konfliktlarni ko‘rish va hal qilish kabilarni bajarishni mashq qilish ham foydali bo‘ladi.

Keling, avval sayt orqali, keyin GUI-mijozlar va buyruq satri interfeysi (CLI) orqali ishlashni qadam-baqadam bosib o‘tamiz.

Repozitoriydagi fayllarni ko‘rish

GitHub repozitoriyda fayllarni ko‘rish uchun tanlash

Ko‘p hollarda manba kodlarini yuklashdan ko‘ra ularni ko‘zdan kechirib chiqish qulayroq ekanini bilsangiz kerak. Ana shunday oddiy operatsiyalar uchun kompyuter dasturi shart emas, fayllarni veb-versiyada ham tezda ochish mumkin (kod ham, suratlar ham saqlanib qoladi). Uni ko‘rish uchun shunchaki ustiga bosish kifoya!

Bu rasmda kodli faylni ko‘rish tasvirlangan. Hammasi risoladagidek: qatorlar raqamlangan, sintaksisga rang berilgan

Bu yerda esa GUI uchun kerak bo'lgan va repozitoriyga joylangan rasm ko‘rsatilgan

Repozitoriylarni topish va o‘qish

Begona repozitoriylarni koʻzdan kechirish vaqt oʻtkazishning ajoyib usuli boʻlsa ham, birinchi navbatda bu sizga yordam beradigan foydali uskunalarni topish usuli hamdir. Ilovangizga qoʻshishni xohlaydigan koʻplab funksiyalarni allaqachon kimdir ishlab chiqqan boʻlishi mumkin, sizga esa faqat GitHub repozitoriylardan shunday loyihalarni izlash qolgan xolos.

Qidiruvning boshqa holatlari kabi kerakli repozitoriyga qidiruv saytlari yoki GitHub'dagi ichki qidiruv tizimi orqali oʻtishingiz mumkin.

Misol uchun: bu Telegram Android ilovasining rasmiy GitHub repozitoriysi. Detallarga qaraymiz: qisqa tavsif – About (Telegram for Android source), ikkita branch (master va dev), boshqaruv tarmogʻida 462 kommit, 227 pull request, oxirgi reliz 6 kun avval, litsenziya GPL 2.0, 1,2 ming kuzatuvchi, 7,1 ming fork va 21 ming qaydlarga qoʻshish holati

Bu yerda nimaga e'tiborni qaratish muhim. Yuqoridagi ekran tasvirining ba'zi punktlarini koʻrib chiqamiz.

Birinchidan, eng anigʻi bu – bosh sahifadagi tavsif. Aynan unda bizning asosiy savolimizga javob bor – bu nima va bizga qanday yordam bera oladi?

Aniqroq aytadigan boʻlsak, repozitoriylarda ikkita tavsif boʻladi: qisqa (yuqori oʻng tomonda bir abzaslik) va toʻliq (oʻrtada, fayllar roʻyxati ostida).

Agar repozitoriy sizga foydali boʻlib koʻrinsa, endi uni qay tarzda ishlata olishingizni aniqlash kerak boʻladi, bu esa koʻrsatilgan litsenziya muallifiga bogʻliq boʻladi.

Shundan soʻng yangilanishlar chastotasi sizga mos kelishini baholashingiz kerak, buni **Releases** boʻlimida qilish mumkin (havolasi qisqa tavsifdan keyin keltiriladi). Albatta, bu vaziyatga bogʻliq: qayerdadir toʻrt yildan beri yangilanmagan uskuna kerak boʻlishi mumkin, lekin koʻpincha kichik boʻlsada quvvatlash, ya'ni hech boʻlmaganda bir necha oyda bir marta yangilanish chiqishi kerak boʻladi.

Kommitlar soni hamda tarkibini koʻrish va bilish ham muhim – bosiladigan hisoblagich fayllar roʻyxati ustida, oʻng tomonda boʻladi.

Nihoyat, jamiyat ichida loyihaning mashhurligini bildiradigan belgilar soniga ham qarash kerak. Buni kuzatuvchi, fork va yulduzchalar (layk va repostlar analogi desa ham bo‘ladi) soni bilan o‘lchash mumkin.

Biroz pastroqda, o‘ng tomonda eng qiziq narsalardan biri: repozitoriyda ishlatilayotgan dasturlash tillari tahlili joylashgan. Yuqoridagi ekran tasviridagi repozitoriyda bu to‘plam shunday ko‘rinishda bo‘ladi.

Telegram ilovasi Android manba repozitoriysiga yuklangan kodlar yozilgan dasturlash tillari

Umuman olganda, o‘zingiz foydalanadigan mashhur ilovalar repozitoriysini o‘rganish va yoqqanini yulduzcha bilan belgilash yaxshi ish. Bu faqat qiziqarli jarayon emas, vaqt o‘tishi bilan GitHub qiziqishlaringizni o‘rganib chiqadi va Explore blokida sizga yoqishi mumkin bo‘lgan narsalarni tavsiya qilishni boshlaydi.

Misol: VK GitHub repozitoriysi

Branch yaratish

№1 — koshka alternativ tarmog‘ini yaratamiz

Yangi branch yaratish juda oson:

New branch katta yashil tugmasini bosamiz.

Yangi branch nomini kiritamiz.

Nusxalash uchun asosiy branchni tanlaymiz (ya’ni **main** branch, chunki hozircha boshqa branch yo‘q).

Create branch’ga bosamiz.

Branchlararo o‘tish va nizolarni hal qilish

Bu holatda avval nima nazarda tutilayotganini aniqlash kerak: agar muqobil branchga shunchaki kirish zarur bo‘lsa, buni repozitoriy branchlari ro‘yxatida ham qilsa bo‘ladi (kerakli havola fayllar ro‘yxati tepasida bo‘ladi).

Xuddi shunday muqobil branch № 2 (sava) ni ham yaratamiz va hammasi oxirida qanday ko‘rinishini kuzatamiz.

Repozitoriy branchlari ro‘yxati: asosiy va ikki muqobil branch

Branchlararo o‘tish haqida boshqa ma’noda gapirsa ham bo‘ladi – bunda muqobil branchdagi o‘zgarishlar asosiy branch bilan birlashtiriladi. Qarang: bizda code.js fayli mavjud, ammo uning tarkibi biroz farq qiladi:

main branch’da JS buyrug‘i console.log (standart tarmoq) shaklida, suratda esa zamonamiz maestrolaridan Rik Estli tasvirlangan;

koshka branchida esa xuddi shu fayl ichida console.log (Koshka!) buyrug‘i, Rikning surati o‘rnida esa kulguli mushuk tasviri bor;

bularning hammasi esa uchinchi **sava** branchi bilan yanada murakkablashadi: unda console.log (“Sova!”) konsol chiqishi va ukki tasviridan foydalanilgan (tabiiyki, barcha branchlardagi suratlar bir xilda pic.jpg deb nomlangan).

Bundan kelib chiqadiki, branchlar orasida konflikt mavjud va muqobil branchlardan istalganini asosiysi bilan birlashtirish mumkin emas! Lekin buning yechimi mavjud: GitHub bizga farqlanuvchi branchlarni solishtirishni taklif qiladi, agar istasak o‘zgarishlarni asosiy branchga kiritish, ya’ni yuqorida aytilgan pull request’ni amalga oshirishni taklif qiladi.

GitHub ba’zi branchlarda o‘zgarishlar borligini shunday ma’lum qiladi

Branchlarni qiyoslash mana shunday ko‘rinishda bo‘ladi (koshka va sava).

main branch’ning koshka branch bilan qiyosi

main branch’ning sava branch bilan qiyosi

Keling, biz sava tarmog‘idagi o‘zgarishlarni qabul qilishga qaror qilib, kichik izoh bilan pull request yaratamiz.

Sava branchidagi o‘zgarishlarni main branch bilan birlashtirish uchun pull request yaratish

Keyin u repozitoriyning pull request’lari ro‘yxatida paydo bo‘ladi va o‘sha yerda o‘zgarish haqidagi mazkur so‘rovning keyingi taqdirini belgilaymiz.

Repozitoriyning tegishli bo‘limida yangi pull request paydo bo‘ldi

Pull request branchlar birlashishi (**merge**) ni tasdiqlab, to‘la qabul qilish yoki so‘rovni yopib (**Close pull request**), rad etish mumkin.

1-qadam. Pull request’ni ko‘rib chiqish

2-qadam. Pull request’ni ko‘rib chiqish

Shunda ham **main** branch’ni repozitoriy sozlamalarida kerakli joylarini belgilab, o‘zgarishlardan saqlash mumkin (va aynan shu narsa yangi tarmoqlar yaratishda sizga taklif ham qilinadi).

GitHub branchni o‘zgarishlardan himoyalashni taklif qilmoqda

Parametrlarning mazkur qatorini o‘zgartirish uchun o‘z repozitoriyingizda **Settings –> Branches** bo‘limiga kiring. Kerakli sozlamalar Branch protection rules bo‘limida bo‘ladi (tanlash uchun 10 ta variant beriladi).

Repozitoriy tavsifini sozlash

GitHub loyihangizning asosiy tavsifi Readme.md faylida beriladi, uni repozitoriy bilan birgalikda yoki keyinroq ham yaratish mumkin. md kengaytmasi bu – soddalashtirilgan mashhur belgilash tili – Markdown’ning qisqartirilgan ko‘rinishi.

Readme faylining tarkibi repozitoriyning bosh sahifasida ko‘rinadi va bu qanday lohiya ekani, boshqa dasturchilarga qanday foyda berishi va undan qanday foydalanish mumkinligi haqidagi savollarga javob bo‘ladi.

Readme faylni did bilan to‘ldirish uchun GitHub’ning markdown belgilash tili haqidagi qo‘llanmasidan foydalanish mumkin. Bizning misol repozitoriyimizning Readme fayli mana shunday ko‘rinishda bo‘ladi (birinchi va ikkinchi ekran).

GitHub repozitoriyning Readme faylini sozlash

GitHub repozitoriyning Readme faylini sozlash

E’tibor bering: faylni to‘ldirishda istalgan format – turli darajadagi sarlavha, shriftlarni qalin/yotiq shaklda ajratish, rasm, emoji, havola va hokazolardan foydalanish mumkin.

Readme fayli anchagina uzun bo‘lishi mumkin, shunday bo‘lsa-da GitHub katta hajmli hujjat uchun “Viki” yaratishni tavsiya qiladi.

GitHub profilingizdan sayt yaratish

GitHub’da GitHub Pages funksiyasi yordamida saytni hosting qilish mumkin. Bu juda oddiy:

Repozitoriy sozlamalariga kiring.

Code and automation blokida **Pages**’ni tanlang.

Manbani tanlang (**Deploy from a branch**, keyin kerakli tarmoqni).

Save tugmasini bosing.

Sahifani yangilang va sahifa tepasida yangi saytingiz uchun havola paydo bo‘ladi.

GitHub pages’dagi sayt namunasi

Saytni loyiha uchun emas, o‘zingizni tanitish uchun yaratsangiz, tashrif-sayt kodi bilan repozitoriy yarating va unga [username].github.io shaklida nom bering. Bu yerda username – GitHub akkauntingizning nomi bo‘ladi. Bu funksiya haqida batafsil ma’lumotni bu yerdan olishingiz mumkin.

GitHub kompyuter dasturiga GUI-mijozni ulash

Agar sizga mana shu ishlash usuli qulay bo‘lsa, bu yerda ham hammasi oddiy. Keling, shu mavzudagi batafsil qo‘llanmamizni qisqacha takrorlab olamiz:

Mijozni yuklab olib, o‘rnatasiz.

O‘z akkauntingizga kirasiz.

Mavjud repozitoriylar bilan ishlaysiz yoki yangi (lokal) larini yaratasiz.

Windows uchun GitHub Desktop’dagi GUI-mijozning bosh menyusi. Ro‘yxatda yuqorida tasvirlangan repozitorimiz – skillbox_cool ham mavjud

Ilova ishni davom ettirish uchun repozitoriyni lokal mashinaga klonlashni tavsiya qiladi.

GitHub kompyuter dasturida repozitoriyni klonlash

O‘chirilgan GitHub repozitoriydan klonlangan fayllar kompyuteringizdagi maxsus papkada saqlanadi va ularga o‘zgartirish kiritish ham oson – mijoz ularni siz tanlagan kod muharriri/dasturlash muhitida ochadi (aslida hammasini alohida qilish – kerakli manba faylini papkada ochish qulayroq).

Umuman olganda, hammasi bir-ikki-uch kabi oson.

CLI orqali GitHub bilan ishlash

GitHub bilan Windows buyruq satri yoki PowerShell orqali ham ishlasa bo‘ladi. Bu unchalik qiyin emas: avvaliga buyruq satri interfeysini yuklash va o‘rnatish kerak (hujjatlar – bu yerda).

PowerShell’da GitHub CLI bilan ishlash

GitHub CLI uchun buyruqlar gh qisqartmasidan boshlanadi, masalan,gh repo clone.

GUI Git interfeysi

Yana boshqa usul ham mavjud – Git’ning o‘zidan foydalanib, uning o‘z CLI yordamida ishlash. Git alohida yuklanib, o‘rnatiladi, uning o‘zida minimalistik GUI mavjud. Ammo uni terminalda ishlatgan mantiqliroq.

PowerShell’da GitHub CLI bilan ishlash

Bu haqida bilish kerak bo‘lgan eng muhim narsa: Git’da barcha buyruqlar gitso‘zi bilan boshlanadi, so‘ng harakat tipi ko‘rsatiladi: git clone, git merge, git fork va hokazo. Ulardan foydalanish uchun Git o‘rnating, uning buyruqlari bo‘yicha eslatma chop eting va qo‘rqmasdan foydalanishni boshlang.

GitHub-profilni sozlash

Profilni kastomizatsiyalash – dasturchining o‘zini tanitishida muhim qism sanaladi: rezyume, tashrif qog‘ozi va ishlagan loyihalaringiz vitrinasi.

Ish beruvchilar bu ma’lumotni munosib baholaydi, shu sababli o‘z haqingizdagi ma’lumotni to‘ldirib, bir nechta yaxshi repozitoriylaringizni ommaga e’lon qilish kerak (albatta, u yerda hech qanday sirli narsa bo‘lmasa). Aynan ular shaxsiy vitrinangizning bir qismiga aylanadi.

Keling, hozir GitHub’da urfga chiqqan biror mashhur dasturchining profilini ko‘ramiz (ha-ha, GitHub’da shunaqa bo‘lim ham bor).

GitHub talqiniga ko‘ra top-25 dasturchi (material yozilgan vaqtda)

Birinchi o‘rinda qandaydir Stiven Selis. Uning profilida nimalar bor? Ish joyi, sayt va aloqa ma’lumotlari ko‘rsatilgan, statistikada 123 repozitoriy va bir yil davomida 1725 ta o‘zgartirish. Oddiy ko‘z bilan ham bu odam faol va tajribali ekanini bilsak bo‘ladi.

Top dasturchilardan birining GitHub profili (2022-yil noyabr oyida GitHub reytingida 1-o‘rin). Yashil kvadratchalar – muallif faolligini bildiradi

Agar siz hali GitHub’da topga chiqmagan bo‘lsangiz ham, profilni shu tarzda to‘ldirishga harakat qiling: batafsil ma’lumot va loyihalaringizdan iborat qator (ularni

profilingizdagi **Customize your pins** tugmasiga bosib, istaganingizdek mahkamlab chiqishingiz mumkin).

end () o‘rnida

Hammasi ko‘ringanidek murakkab bo‘lmaydi (boshqacha aytadigan bo‘lsak, har bir dasturchi hayoti davomida avval sanchqida yeyishni o‘rganadi, keyin GitHub repozitoriylarni forklaydi).

GitHub’dan hamma foydalanadi: bu dasturlash yo‘nalishi va tilidan qat’i nazar, eng muhim umumiy ko‘nikma sanaladi. Maktabdagi Hayot xavfsizligi asoslari fanidek ana o‘sha git clone ham qachondir hayotingizni saqlab qoladi.

Shu sababli o‘quv kodlarining bekaplari uchun ham GitHub’dan foydalanishni iloji boricha ertaroq boshlash kerak va bu tez orada foydali odatga aylanib qoladi.

Nazorat savollari.

1. NoCode haqida ma’lumot bering.
2. Nocoding texnolgiyasini tushuntiring.
3. Git Hub bilan ishlash qanday amalga oshiriladi?